

Knowledge Representation and Reasoning – Mod. 2

Exercise Sheet 6: RDFS – Solutions

Gian Carlo Milanese

27 May 2025

Acknowledgements

The Geo ontology modeling exercises on Protégé and their solutions are from Matteo Palmonari's RDFS and OWL Hands-on Session for the Artificial Intelligence (MSc Computer Science) and Data Semantics (MSc Data Science) courses of the University of Milano-Bicocca.

Preliminaries

You can use an online validator such as the following to ensure that your Turtle syntax is valid.

- <http://ttl.summerofcode.be/>. Turtle validator.
- <https://www.easyrdf.org/converter>. Here you can convert RDF data between different serializations. An error will be thrown if your syntax is not correct.
- <https://www.w3.org/RDF/Validator/>. Here you can validate RDF data serialized as RDF/XML (you can use the previous converter to convert it to this format). If you select “Triples and Graph” among the Display Result Options you can also see a graph visualization.

RDFS Entailment Patterns

If **C**, **D**, **E** (“classes”), **P**, **Q**, **R** (“relations”), **X** and **Y** (“entities”) are RDF Terms:

	If...	Then...
rdfs2	P rdfs:domain C X P Y	$\implies$ X rdf:type C
rdfs3	P rdfs:range D X P Y	$\implies$ Y rdf:type D
rdfs5	P rdfs:subPropertyOf Q Q rdfs:subPropertyOf R	$\implies$ P rdfs:subPropertyOf R
rdfs7	P rdfs:subPropertyOf Q X P Y	$\implies$ X Q Y
rdfs9	C rdfs:subClassOf D X rdf:type C	$\implies$ X rdf:type D
rdfs11	C rdfs:subClassOf D D rdfs:subClassOf E	$\implies$ C rdfs:subClassOf E

Read [here](#) for a full list of RDFS entailment patterns.

Exercise 1 RDFS: Instances, Classes, Properties

1. Using the following IRIs (and others that you will introduce with the prefix `ex:`, e.g. `ex:bob` or `ex:Person`)

- `rdf:type`
- `rdfs:Class`
- `rdf:Property`
- `rdfs:subClassOf`
- `rdfs:domain`
- `rdfs:range`
- `rdfs:subPropertyOf`

write down a Turtle document that expresses the following:

- Carol is a student.
 - Bob likes the album *The dark side of the moon*, Alice likes *The wall*.
 - Bob has purchased the album *Meddle*, and Alice has purchased the album *Pulse*.
 - Students are people. People are animals.
 - If someone purchases something, then they own it.
 - The property *purchase* relates resources of type *Person* to resources of type *Music album*.
- (a) Specify in your RDF graph which resources are of type `rdfs:Class` and which are of type `rdf:Property`.
 - (b) What inferences can you make?

Solution. See the graph below.

- (c) What if you wanted to add *movies* to your knowledge graph, and express, for instance, that Bob has purchased the movie *Memento*? How would you change your schema?

Solution. If we keep the same schema, adding “`ex:bob ex:purchase ex:Memento`” would cause the inference “`ex:Memento a ex:MusicAlbum`”, which makes no sense.

We would at least need to change “`ex:purchase rdfs:range ex:MusicAlbum`” into something like “`ex:purchase rdfs:range ex:Product`”.

We should also add

```
ex:Movie a rdfs:Class .
ex:Memento a ex:Movie .
```

You can use prefixes, in particular the ones defined below:

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns##> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema##> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema##> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex: <http://example.org/> .
## continue here
```

2. Consider the following triples:

- `P rdfs:range C`
- `P rdfs:range D`

Provide examples of `P`, `C` and `D` for which such statements (taken together) make sense, and examples for which they do not.

Solution. For part 1, for example:

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:Person a rdfs:Class . ex:Animal a rdfs:Class .
ex:Student a rdfs:Class . ex:MusicAlbum a rdfs:Class .

ex:like a rdf:Property .
ex:purchase a rdf:Property .
ex:own a rdf:Property .

ex:Student rdfs:subClassOf ex:Person .
ex:Person rdfs:subClassOf ex:Animal .

ex:purchase rdfs:subPropertyOf ex:own .
ex:purchase rdfs:domain ex:Person ; rdfs:range ex:MusicAlbum .

ex:carol a ex:Student .
ex:alice ex:like ex:TheWall ; ex:purchase ex:Pulse .
ex:bob ex:like ex:DarkSideOfTheMoon ; ex:purchase ex:Meddle .

ex:Meddle a ex:MusicAlbum .
ex:DarkSideOfTheMoon a ex:MusicAlbum .
ex:Pulse a ex:MusicAlbum .

# Answer to 1(b): # Examples of inferences that we can make:
# Since ex:MusicAlbum is the range of ex:purchase:
ex:TheWall a ex:MusicAlbum .
# Since ex:purchase is a subproperty of ex:own
ex:alice ex:own ex:Pulse .
ex:bob ex:own ex:Meddle .
# Since ex:Student is a subclass of ex:Person,
# which is a subclass of ex:Animal:
ex:carol a ex:Person, ex:Animal .
# Since ex:Person is the domain of ex:purchase,
# and ex:Person is a subclass of ex:Animal:
ex:alice a ex:Person, ex:Animal .
ex:bob a ex:Person, ex:Animal .
```

For part 2:

- While nothing prevents you to write, e.g.
 ex:bob ex:like ex:billy .
 ex:like rdfs:range ex:Dog, ex:Cat .
this would imply
 ex:billy a ex:Dog, ex:Cat .
which does not make sense.

- A statement that would be ok could be:

```
ex:like rdfs:range ex:ItalianCheese, ex:BlueCheese .
ex:bob ex:like ex:gorgonzola .
```

Exercise 2 RDFS Inferences 1

Consider the following RDF knowledge graph G and schema S (below):

```
@prefix : <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
# ----- #
#                               #
#                               #
# ----- #
```

```
:London a :City .
:Glasgow a :City .
:England a :Country .
:UK      a :SovereignCountry .
```

```
:London      :hasCountry :England .
:Manchester   :hasCountry :England .
:Glasgow      :hasCountry :Scotland .
:Aberdeen     :hasCountry :Scotland .
```

```
:England :partOf :UK .
:Scotland :partOf :UK .
```

```
# ----- #
#                               #
#                               #
# ----- #
```

```
:hasCountry rdfs:domain :City ; rdfs:range :Country .
```

```
:City          rdfs:subClassOf :Region .
:Country        rdfs:subClassOf :Region .
:SovereignCountry rdfs:subClassOf :Region .
```

1. List all the statements with predicate `rdf:type` (you can use `a` instead of `rdf:type`) that are not in G and that can be inferred based on S .
2. Specify if “`:Aberdeen :partOf :Scotland .`” is an inference from $S \cup G$ and, if not, propose a statement in RDFS to obtain such inference.
3. Specify the statements not in G that can be inferred by adding the following two statements to S as specified above (without changes proposed to answer the previous question):

```
:partOf rdfs:domain :City ; rdfs:domain :Country .
```

Discuss which benefits and/or drawbacks are introduced by this addition.

Solution. 1. The class-related triples listed below can be inferred;

```

:London a :Region .
:Glasgow a :Region .
:England a :Region .
:UK a :Region .
:Manchester a :City , :Region .
:Aberdeen a :City , :Region .
:Scotland a :Country , :Region .

```

2. `:Aberdeen :partOf :Scotland .` is not an inference; in order to infer this statement the following triple must be added to S : `:hasCountry rdfs:subPropertyOf :partOf .`
3. The following statements can be inferred (* means not in G):

```

:Scotland a :Country* , :City* .
:England a :Country , :City* .

```

The drawback is that intuitively something cannot be a `:Country` and a `:City` at the same time. However, in RDFS we cannot express that two classes are disjoint. In OWL we could say `:Country owl:disjointWith :City`, which will then give rise to a contradiction.

Exercise 3 RDFS Inferences 2

Consider the following RDF knowledge graph G and schema S :

```

@prefix : <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

```

```

# ----- #
#                               #
#                               #
# ----- #

```

```

:Marcelo :team :RealMadrid .

```

```

:Messi :offenseTeam :BarcelonaFC .
:Alves :defenseTeam :BarcelonaFC .

```

```

:BarcelonaFC :location :Barcelona .
:Barcelona :location :Spain .

```

```

# ----- #
#                               #
#                               #
# ----- #

```

```

:location rdfs:range :Place .

```

```

:offenseTeam rdfs:subPropertyOf :team .
:defenseTeam rdfs:subPropertyOf :team .
:team rdfs:subPropertyOf :memberOf .

```

```

:memberOf rdfs:domain :Person ; rdfs:range :Organization .

```

```

:offenseTeam rdfs:domain :Forward , :SoccerPlayer ;
              rdfs:range  :SoccerTeam .
:defenseTeam rdfs:domain :Defender , :SoccerPlayer ;
              rdfs:range  :SoccerTeam .
:team        rdfs:domain :SoccerPlayer , :Person ;
              rdfs:range  :SoccerTeam .

```

1. Fill in Table 1 by listing all entity types inferred from S not in G (i.e., statements that are not in G and that can be inferred using S as a schema). The first row is provided as a hint.

<i>Entity</i>	<i>:Type_1, :Type_2, ...</i>
:Marcelo	:SoccerPlayer, :Person
:Messi	:Forward, :SoccerPlayer, :Person
:Alves	:Defender, :SoccerPlayer, :Person
:RealMadrid	:SoccerTeam, :Organization
:BarcelonaFC	:SoccerTeam, :Organization
:Barcelona	:Place
:Spain	:Place

Table 1: Inferred types.

2. Fill in Table 2 by listing every relation inferred from S and not in G (i.e., statements that are not in G and that can be inferred using S as a schema). The first row is partially filled as a hint

<i>Property</i>	<i>(:subject_1, :object_1), (:subject_2, :object_2), ...</i>
:team	(:Alves, :BarcelonaFC), (:Messi, :BarcelonaFC)
:memberOf	(:Alves, :BarcelonaFC), (:Marcelo, :RealMadrid), (:Messi, :BarcelonaFC)

Table 2: Inferred relations.

3. Specify which inferences - if any - are added if the axioms below are introduced.

```

:Defender rdfs:subClassOf :SoccerPlayer .
:Forward rdfs:subClassOf :SoccerPlayer .
:SoccerPlayer rdfs:subClassOf :Person .
:SoccerTeam rdfs:subClassOf :Organization .

```

4. Is there any RDFS axiom (i.e., a schema-level axiom using some RDFS predicate) that can be added to S to infer the following statement? Briefly motivate your answer.

```

:Messi :location :Spain .

```

Solution. 1. See Table 1.

2. See Table 2.

3. No inference is added.

4. It is not possible: we'd need composition and transitivity (Messi plays in Barcelona FC, which is located in Barcelona, which is located in Spain) which are not supported in RDFS.

Exercise 4 RDFS Inferences in SPARQL

Go to <https://dbpedia.org/sparql>.

1. Run a SPARQL query to find all the types of `dbr:Alfred_Hitchcock` (or some other IRI of your choice), i.e., all the classes `?c` such that `dbr:Alfred_Hitchcock` is an instance (`rdf:type`) of `?c`. Try different versions of this query by simulating RDFS inference patterns (e.g., use `rdf:type/rdfs:subClassOf*`), and without doing so. Do you get different results? Why?

Solution.

A query that does not simulate RDFS reasoning would be:

```
select distinct ?c where {
  dbr:Alfred_Hitchcock a ?c
}
```

A query that simulates the rules `rdfs9` and `rdfs11` would be:

```
select distinct ?c where {
  dbr:Alfred_Hitchcock a/rdfs:subClassOf* ?c
}
```

A query that also simulates `rdfs2` and `rdfs3` would be:

```
select distinct ?c where {
  { dbr:Alfred_Hitchcock ?p ?y . ?p rdfs:domain/rdfs:subClassOf* ?c }
  union
  { ?y ?p dbr:Alfred_Hitchcock . ?p rdfs:range/rdfs:subClassOf* ?c }
  union
  { dbr:Alfred_Hitchcock a/rdfs:subClassOf* ?c }
}
```

The reason why you will probably *not* get more results with the second query is that in DBPedia many `rdfs:subClassOf`-related inferences are already *materialized*, i.e., they have already been added to the DBPedia Knowledge graph. This does not mean that SPARQL is performing any reasoning. SPARQL is still doing basic graph pattern matching. However, since the inferred triples have already been added to DBpedia, SPARQL is also matching these triples.

2. Run a SPARQL query that *constructs* the RDF graph representing the class hierarchy of all subclasses of `dbo:Animal`. Use the validators/converters linked in the Preliminaries to see a graph representation of your output.

Solution.

```
construct { ?c rdfs:subClassOf ?d }
where { ?c rdfs:subClassOf ?d . ?d rdfs:subClassOf* dbo:Animal }
```

3. Run a query that simulates the patterns `rdfs2` and `rdfs3` to infer additional types `?c` for resources `?x` of type `dbo:Person` (retrieve the bindings for `?x` and `?c` or construct a graph of

inferences; limit the number of results for faster query times).

Solution.

```
select distinct ?x ?c where {  
  ?x a dbo:Person .  
  ?x ?p ?o . ?p rdfs:domain ?c.  
} limit 100
```

4. Same as the previous query, but exclude types that are already available in DBpedia (i.e., only return new types that are not explicitly available in DBpedia).

Solution.

```
select distinct ?x ?c where {  
  ?x a dbo:Person .  
  ?x ?p ?o . ?p rdfs:domain ?c.  
  filter not exists { ?x a ?c } } limit 100
```

Exercise 5 Protégé: Geo Ontology Modeling Level 1

For this part use only features that come from RDF and RDFS (type, subclass, subproperty, domain, range). In other words, don't use property restrictions (existential or universal), property characteristics such as symmetry, etc., class definitions (equivalent to).

1. Introduce the following classes:
 - Continent, Country, Region, City
 - Organization, Company, Non-governmental Organization (NGO).
2. For Country, define subclasses on a geographical basis (e.g., EuropeanCountry), to support grouping of countries based on the continent in which they appear.
3. Define a few individuals, as suggested below (but create some of them as instances of `owl:Thing` for the moment: we will infer their classes later):
 - Continents: all;
 - Countries: Italy, France, Netherlands;
 - Regions: Lombardy, Lazio, Piedmont;
 - Cities: Milan, Rome, Paris, Lyon, Amsterdam, Utrecht;
 - Companies: FIAT, SAP;
 - NGOs: Emergency, MSF.
4. Introduce properties to model the relationships between individuals of their respective classes (in the Tbox and in the Abox):
 - A property to link the *contained* geographical entities to those that contain them.
 - A specific property to link cities to the countries they are *capital of*
 - A property to specify the *population* of countries (based on Wikipedia)
 - A property `hasHeadquartersIn` to specify the country where an organization has its headquarters
 - A property `hasOfficesIn` to specify where an organization has offices, including headquarters, but not limited to headquarters

5. Introduce at least two Abox assertions for each property (e.g., Roma is the capital of Italy).
6. Introduce at least two annotations using `rdfs:label`
 - Suggestion: use MSF for Medici Senza Frontiere and add "`Medici Senza Frontiere`" as its label; use NGO for Non-governmental organization and add the full name as its `rdfs:label`.
7. Specify the meaning of the properties using only RDFS constructs (domain/range restrictions).
8. Use a reasoner (e.g., Hermit) and compute the inferences making sure that the Abox is defined in such a way that you can infer at least one-two classes for the instances, which were not explicitly defined (e.g., infer the type City for some city).

Bonus: a query that covers (almost) all RDF and RDFS entailment patterns

The following query covers (almost) all **RDF** and **RDFS entailment patterns**. To use it to (iteratively) materialize inferences in an RDF graph, replace **CONSTRUCT** with **INSERT**.

```
CONSTRUCT { ?s ?p ?o } WHERE {
  ## RDF entailment
  # rdfD1 is skipped
  # rdfD2
  { ?x ?s ?y . bind ( rdf:type as ?p ) bind ( rdf:Property as ?o ) }
  ## RDFS entailment
  # rdfs1 is skipped
  union # rdfs2
  { ?pp rdfs:domain ?o . ?s ?pp ?x . bind ( rdf:type as ?p ) }
  union # rdfs3
  { ?pp rdfs:range ?o . ?x ?pp ?s . bind ( rdf:type as ?p ) }
  union # rdfs4a
  { ?s ?pp ?oo . bind ( rdf:type as ?p ) bind ( rdfs:Resource as ?o ) }
  union # rdfs4b
  { ?ss ?pp ?s . bind ( rdf:type as ?p ) bind ( rdfs:Resource as ?o ) }
  union # rdfs5
  { ?s rdfs:subPropertyOf/rdfs:subPropertyOf ?o .
    bind ( rdfs:subPropertyOf as ?p ) }
  union # rdfs6
  { ?s rdf:type rdf:Property . bind ( rdfs:subPropertyOf as ?p )
    bind ( ?s as ?o ) }
  union # rdfs7
  { ?q rdfs:subPropertyOf ?p . ?s ?q ?o }
  union # rdfs8
  { ?s rdf:type rdfs:Class . bind ( rdfs:subClassOf as ?p )
    bind ( rdfs:Resource as ?o ) }
  union # rdfs9
  { ?c rdfs:subClassOf ?o . ?s rdf:type ?c . bind ( rdf:type as ?p ) }
  union # rdfs10
  { ?s rdf:type rdfs:Class .
    bind ( ?s as ?o ) bind ( rdfs:subClassOf as ?p ) }
  union # rdfs11
  { ?s rdfs:subClassOf/rdfs:subClassOf ?o . bind ( rdfs:subClassOf as ?p ) }
  union # rdfs12
  { ?s rdf:type rdfs:ContainerMembershipProperty .
    bind ( rdfs:subPropertyOf as ?p )
    bind ( rdfs:member as ?o ) }
  union # rdfs13
  { ?s rdf:type rdfs:Datatype . bind ( rdfs:subClassOf as ?p )
    bind ( rdfs:Literal as ?o ) }
}
```